

Modellauflösung im RAG-Projekt – Workflow von .env bis ChatOpenAI

Zusammenfassung des vollständigen Wegs, wie das Chat-Modell vom `.env`-Eintrag bis zur fertigen LLM-Instanz bestimmt wird. Fünf Stationen, drei Dateien.

Reihenfolge der Auflösung, von der schwächsten zur stärksten Quelle:

Argument > CHAT_MODEL aus .env > Katalog-Default (default=True)

Übersicht

.env → Settings → resolve_target() → ModelRegistry →
ChatOpenAI
(providers.py) (config.py) (providers.py) (models.py)

Wo das Modell gesetzt werden kann

Ebene	Ort	Wirkung
-----	-----	
Grundeinstellung	models.py, default=True	wenn nichts anderes gesetzt
Projektkonfig	.env, CHAT_MODEL	bei jedem Start
Einzellauf (Code)	ChatBot(model=...)	nur dieser Bot
Einzellauf (CLI)	--model ...	nur dieser Aufruf

Station 1 – Die .env wird gelesen

Auslöser ist der erste Aufruf von `get\_settings()` in `config.py`. Davor erledigt `load\_environment()` das Einlesen:

```
def load_environment(dotenv_path: Path | None = None) -> bool:
    target = dotenv_path or PROJECT_ROOT / ".env"
    return load_dotenv(dotenv_path=target, override=False)
```

`override=False` heißt: Eine bereits in der Prozessumgebung gesetzte Variable gewinnt gegen die .env. In Docker oder CI ist das gewollt.

Gesucht wird die Datei über PROJECT_ROOT (Elternordner des Pakets):

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

Beispielpfad in diesem Projekt: C:\AZAV-ML\RAG-projects

Danach werden die Werte in `Settings` überführt. Für das Modell sind zwei Felder relevant:

```
chat_provider: ChatProvider = ChatProvider.OPENROUTER
chat_model: str | None = None
```

FILTER 1 greift hier: Ist CHAT_MODEL leer oder fehlt, normalisiert der Validator den Wert:

```
@field_validator("chat_model", mode="before")
@classmethod
def _blank_model_to_none(cls, value):
    if isinstance(value, str) and not value.strip():
        return None
    return value
```

Aus `CHAT\_MODEL=` wird also None – die Zeile zählt fortan als "nicht gesetzt". Ohne diesen Validator würde ein leerer String später als Modellkennung behandelt und einen UnknownModelError auslösen.

Ergebnis dieser Station:

```
chat_provider = ChatProvider.OPENROUTER
chat_model    = "openrouter:z-ai/glm-5.3"
               (bzw. None, wenn die Zeile leer ist)
```

Station 2 – Die Priorisierung in resolve_target()

Jetzt entscheidet sich, welche Quelle gewinnt. Die Regel steht in einer einzigen Zeile in providers.py:

```
raw = (model or settings.chat_model or "").strip()
```

Von links nach rechts:

```
Argument (Code/CLI) → CHAT_MODEL aus .env → ""
```

Danach teilt sich der Weg:

```
if raw:
    # Weg A: eine Kennung liegt vor -> Station 3
else:
    # Weg B: nichts gesetzt          -> Station 4
```

Station 3 – Weg A: Auflösung gegen den Katalog

Liegt eine Kennung vor, prüft der Resolver zuerst das Präfix:

```
prefix = raw.partition(":")[0].lower() if ":" in raw else None
if prefix and prefix in {p.value for p in ChatProvider}:
    target = model_registry.resolve(raw)      # Präfix entscheidet
    provider = target.provider
else:
```

```

provider = settings.chat_provider          # aktiver Provider zuerst
try:
    target = model_registry.get(f"{provider.value}:{raw}")
except UnknownModelError:
    target = model_registry.resolve(raw)    # dann Gesamtkatalog
    provider = target.provider

```

Drei Fälle, die daraus folgen:

Eingabe	Weg	Ergebnis
-----	-----	

openrouter:z-ai/glm-5.3	Präfix erkannt	direkt bei
OpenRouter		
z-ai/glm-5.3	kein Präfix	erst
openrouter:..., gefunden		
gpt-5.6-luna	kein Präfix, bei beiden	UnknownModelError
	Providern vorhanden	(nicht
entscheidbar)		

Das Nachschlagen selbst passiert in `ModelRegistry.get()` über ein Wörterbuch, dessen Schlüssel die `qualified_id` ist:

```

def get(self, qualified_id: str) -> ModelInfo:
    found = self._by_qualified.get(qualified_id.strip().lower())
    if found is None:
        raise UnknownModelError(qualified_id, self.suggest(qualified_id))
    return found

```

Die `qualified_id` hat die Form "<provider>:<id>", zum Beispiel:

```
openrouter:z-ai/glm-5.3
```

Gefüllt wird `_by_qualified` im Konstruktor von `ModelRegistry` aus `RAW_MODELS` – also beim Import von `models.py`.

Station 4 – Weg B: Der Katalog-Default

Ist `raw` leer, greift der Default:

```

provider = settings.chat_provider
target = model_registry.default_for(provider)

```

`default_for()` durchsucht die Einträge des Providers nach dem Flag:

```

def default_for(self, provider: ChatProvider) -> ModelInfo:
    for model in self.for_provider(provider):
        if model.default:
            return model
    # Kein explizites Default markiert -> erster Eintrag des Providers.
    candidates = self.for_provider(provider)

```

```

        if not candidates:
            raise LookupError(f"Kein Modell fuer Provider {provider.value} im
Katalog")
        return candidates[0]

```

Die Flags im Katalog (models.py):

```

default=True    bei "openai/gpt-5.6-luna"    in OPENROUTER_MODELS
default=True    bei "gpt-4o-mini"           in OPENAI_MODELS

```

Bei CHAT_PROVIDER=openrouter und leerem CHAT_MODEL ergibt das:

```
openrouter:openai/gpt-5.6-luna
```

Station 5 – Konsistenzprüfung und Client-Bau

Vor dem Bau des Clients prüft resolve_target() zwei Dinge.

Erstens, ob Modell und Provider zusammenpassen:

```

    if target.provider is not provider:
        raise ProviderError(
            f"Modell {target.qualified_id!r} gehoert zu
{target.provider.value!r}, "
            f"aktiv ist aber {provider.value!r}. "
            f"Setze CHAT_PROVIDER={target.provider.value} oder waehle ein "
            f"Modell dieses Providers."
        )

```

Zweitens, ob der Key des Providers existiert:

```

def _assert_key_present(provider: ChatProvider, settings: Settings) -> None:
    key_field = provider.key_field
    if getattr(settings, key_field) is None:
        raise MissingCredentialsError(
            f"{provider.env_var} is not set. "
            f"Lege den Key in der .env an: {provider.env_var}=..."
        )

```

Dann steht das Ergebnis fest – ein ResolvedTarget:

```

@dataclass(frozen=True, slots=True)
class ResolvedTarget:
    provider: ChatProvider
    model: ModelInfo
    base_url: str
    temperature: float | None
    max_tokens: int

```

Hier wird temperature auf None gesetzt, wenn das Modell den Parameter nicht unterstützt (models.py: supports_temperature=False, z. B. bei gpt-5-pro).

build_llm() übersetzt das in Client-Parameter:

```
kwargs = {
    "model": target.model.id,          # ID aus ModelInfo, ohne Präfix
    "base_url": target.base_url,       # https://openrouter.ai/api/v1
    "api_key": settings.api_key_value, # EINZIGE Entschlüsselung im Projekt
    "max_tokens": target.max_tokens,
    "timeout": settings.request_timeout,
    "streaming": streaming,
}
if target.temperature is not None:
    kwargs["temperature"] = target.temperature
```

Bei OpenRouter kommen zwei optionale Kopfzeilen dazu (HTTP-Referer, X-Title).

Abschließend:

```
return ChatOpenAI(**kwargs)
```

Das ist die letzte Entscheidung im gesamten Ablauf.

Die Kette in einem Durchlauf

1	load_environment()	.env geladen	(PROJECT_ROOT/.env)
2	Settings(...)	chat_model gesetzt	[leer -> None per
Validator]			
3	resolve_target()	raw = model or settings.chat_model or ""	
4	Präfix erkannt?	ja -> ModelRegistry.resolve() nein -> get(f"{provider}:{raw}") -> Fallback Gesamtkatalog	
5	[raw leer?]	default_for(provider) -> Eintrag mit default=True	
6	Provider-Konsistenz	target.provider is provider?	
7	Key vorhanden?	getattr(settings, provider.key_field) is not None	
8	ResolvedTarget	provider, model, base_url, temperature,	
max_tokens			
9	build_llm()	ChatOpenAI(model=target.model.id, base_url=..., api_key=..., ...)	

Fehlerfälle an den einzelnen Stationen

Fehler	Typ	Ausgelöst in
-----	-----	-----
Modell nicht im Katalog	UnknownModelError	Station 3
Modell ohne Präfix, mehrdeutig	UnknownModelError	Station 3
Modell passt nicht zu Provider	ProviderError	Station 5
Key des Providers fehlt	MissingCredentialsError	Station 5

Der mehrdeutige Fall im Katalog

Ein einziger Eintrag existiert bei beiden Providern:

```
openrouter:openai/gpt-5.6-luna
openai:gpt-5.6-luna
```

Ohne Präfix ist die Kennung nicht auflösbar. `registry.resolve()` wirft dann einen `UnknownModelError` und schlägt beide qualifizierten Formen als Kandidaten vor. Mit Präfix ist der Fall eindeutig.

Prüfbefehle (ohne Netzwerk)

```
# Gesetztes Modell vs. Default auflösen
uv run python -c "
from rag_project.models import registry
from rag_project.config import ChatProvider
print('gesetzt :', registry.get('openrouter:z-ai/glm-5.3').qualified_id)
print('Default :',
registry.default_for(ChatProvider.OPENROUTER).qualified_id)
"

# Gesamte Auflösungskette
uv run python -c "
from rag_project.providers import resolve_target
for k in ('openrouter:z-ai/glm-5.3', 'openai:gpt-4o-mini'):
    print(k, '->', resolve_target(k).summary())
print('Default ->', resolve_target().summary())
"

# Aktive Konfiguration (Keys nur als Fingerabdruck)
uv run rag-project --show-config

# Katalog mit Markierung des Defaults
uv run rag-project --models
```